

AI ASSISTANT CODING

ASSIGNMENT-11.5

Name: B.Thanmai

Enrollment no: 2303A52126

Batch: 33

Semester: VI

Branch: CSE(AIML)

Lab 11 – Data Structures with AI: Implementing Fundamental Structures

Task Description #1 – Stack Implementation

Task: Use AI to generate a Stack class with push, pop, peek, and is_empty methods.

Sample Input Code: class

Stack:

pass

Expected Output:

- A functional stack implementation with all required methods and docstrings.

```
"""Prompt:
Generate a Python class named Stack that implements the stack data
structure using a list. The class should include the following
methods:
push(item) - to add an element to the top of the stack
pop() - to remove and return the top element
peek() - to return the top element without removing it
is_empty() - to check whether the stack is empty
Include proper docstrings for the class and each method. Handle edge
cases such as popping from an empty stack."""
```

```
main.py
10 class Stack:
11     """
12     A class to represent a stack data structure.
13     Uses a Python list to store stack elements.
14     """
15     def __init__(self):
16         """Initialize an empty stack."""
17         self.items = []
18
19     def push(self, item):
20         """
21         Add an element to the top of the stack.
22         param item: Element to be added to the stack
23         """
24         self.items.append(item)
25
26     def pop(self):
27         """
28         Remove and return the top element of the stack.
29         return: Top element of the stack
30         raises IndexError: If the stack is empty
31         """
32         if self.is_empty():
33             raise IndexError("Pop from empty stack")
34         return self.items.pop()
35
36     def peek(self):
37         """
38         Return the top element without removing it.
39         return: Top element of the stack
40
41 main.py
42     """
43     return self.items.pop()
44
45     def peek(self):
46         """
47         Return the top element without removing it.
48         return: Top element of the stack
49         raises IndexError: If the stack is empty
50         """
51         if self.is_empty():
52             raise IndexError("Peek from empty stack")
53         return self.items[-1]
54
55     def is_empty(self):
56         """
57         Check whether the stack is empty.
58         return: True if stack is empty, False otherwise
59         """
60         return len(self.items) == 0
61
62 if __name__ == "__main__":
63     stack = Stack()
64
65     stack.push(10)
66     stack.push(20)
67     stack.push(30)
68
69     print("Top element:", stack.peek()) # 30
70     print("Popped element:", stack.pop()) # 30
71     print("Is stack empty?", stack.is_empty()) # False
72
73     """
74     Top element: 30
75     Popped element: 30
76     Is stack empty? False
77
78     """
79     """
80     Code Execution Successful
81     """
```

Task Description #2 – Queue Implementation Task:

Use AI to implement a Queue using Python lists.

Sample Input Code:

```
class Queue:
```

```
pass
```

Expected Output:

- FIFO-based queue class with enqueue, dequeue, peek, and size methods.

""" Prompt:
Generate a Python class named Queue that implements a FIFO (First-In-First-Out) queue using a Python list. The class should include the following methods:

enqueue(item) - to add an element to the rear of the queue

dequeue() - to remove and return the front element

peek() - to return the front element without removing it

size() - to return the number of elements in the queue """

```
main.py  Run Output Clear
11 class Queue:
12     """
13     A class to represent a FIFO (First-In-First-Out) queue.
14     Uses a Python list to store queue elements.
15     """
16     def __init__(self):
17         """Initialize an empty queue"""
18         self.items = []
19
20     def enqueue(self, item):
21         """
22         Add an element to the rear of the queue.
23         (parameter: Element to be added to the queue)
24         """
25         self.items.append(item)
26
27     def dequeue(self):
28         """
29         Remove and return the front element of the queue.
30         (parameter: None)
31         """
32         return self.items.pop(0)
33
34     def peek(self):
35         """
36         Return the front element without removing it.
37         (parameter: None)
38         """
39         return self.items[0]
40
41     def size(self):
42         """
43         Return the number of elements in the queue.
44         (parameter: None)
45         """
46         return len(self.items)
47
48     def is_empty(self):
49         """
50         Check whether the queue is empty.
51         (parameter: None)
52         """
53         return len(self.items) == 0
54
55 if __name__ == "__main__":
56     queue = Queue()
57
58     queue.enqueue(10)
59     queue.enqueue(20)
60     queue.enqueue(30)
61
62     print("Front element:", queue.peek()) # 10
63     print("Dequeued element:", queue.dequeue()) # 10
64     print("Queue size:", queue.size()) # 2
```

Output

```
Front element: 10
Dequeued element: 10
Queue size: 2
Code Execution Successful
```

Task Description #3 – Linked List

Task: Use AI to generate a Singly Linked List with insert and display methods.

Sample Input Code:

class Node:

pass

LinkedList:

pass

Expected Output:

- A working linked list implementation with clear method documentation

```
""" Prompt:
Generate a Python implementation of a Singly Linked List.
Create a Node class to store data and a reference to the next node.
Create a LinkedList class with the following methods:

insert(data) - to insert a new node at the end of the list
display() - to display all elements of the linked list
Include clear docstrings for all classes and methods. """
```

```
1 class Node:
2     """
3     Node class represents a single element in a singly linked list.
4     Attributes:
5     data : Stores the value of the node.
6     next : Reference to the next node in the list (default is None).
7     """
8     def __init__(self, data):
9         """
10        Initialize a new node with given data.
11        """
12        self.data = data
13        self.next = None
14
15 class LinkedList:
16     """
17     LinkedList class represents a singly linked list.
18     Attributes:
19     head : Points to the first node of the linked list.
20     """
21     def __init__(self):
22         """
23         Initialize an empty linked list.
24         Initially, head is set to None.
25         """
26         self.head = None
27
28     def insert(self, data):
29         """
30        Insert a new node at the end of the linked list.
31        """
32        self._insert(data)
33
34    def _insert(self, data):
35        """
36        Insert a new node at the end of the linked list.
37        """
38        if self.head is None:
39            self.head = Node(data)
40            return
41
42        current = self.head
43        while current.next is not None:
44            current = current.next
45
46        current.next = Node(data)
```

```

14: data = The value to be inserted into the list.
15: If the list is empty, the new node becomes the head.
16: Otherwise, traverse to the end and attach the new node.
17:
18: new_node = Node(data)
19: A if head is None, then initialize the head
20: if self.head is None:
21:     self.head = new_node
22:     return
23: A traverse to the end of the linked list
24: current = self.head
25: while current.next:
26:     current = current.next
27: A Link the last node to the new node
28: current.next = new_node
29: self.display()
30:
31: Display all elements of the linked list.
32: Traverse from head to the last node
33: and prints each element followed by '\n'
34:
35: current = self.head
36: if current is None:
37:     print('Linked list is empty.')
38:     return
39: while current:
40:     print(current.data, end=' ')
41:     current = current.next
42:
43: print('\n')
44: if __name__ == '__main__':
45:     linked_list = LinkedList()
46:
47:     linked_list.insert(10)
48:     linked_list.insert(20)
49:     linked_list.insert(30)
50:     linked_list.display()

```

Task Description #4 – Hash Table

Task: Use AI to implement a hash table with basic insert, search, and delete methods.

Sample Input Code: class

HashTable:

pass

Expected Output:

- Collision handling using chaining, with well-commented methods

```

1 """Task Description #4 - Hash Table
2 Task: Use AI to implement a hash table with basic insert, search, and delete
3 methods.
4 Sample Input Code:
5 class HashTable:
6     pass
7 Expected Output:
8 • Collision handling using chaining, with well-commented methods. """

```

```

1 class HashTable:
2     """
3     HashTable implementation using chaining for collision resolution.
4     Attributes:
5         size: Size of the hash table.
6         table: List of buckets (each bucket is a list).
7     """
8     def __init__(self, size=10):
9         """
10        Initialize the hash table with given size.
11        Parameters:
12            size (int): Number of buckets in the hash table.
13        """
14        self.size = size
15        self.table = [[] for _ in range(size)]
16
17    def search(self, key):
18        """
19        Returns whether to remove hash code for a given key.
20        Parameters:
21            key: Key to hash.
22        Returns:
23            bool: True if bucket is empty, False otherwise.
24        """
25        return self.table[key] == []
26
27    def insert(self, key, value):
28        """
29        Insert a key-value pair into the hash table.
30        Handles collisions using chaining.
31        If the key already exists, update its value.
32        """
33        index = self._hash(key)
34        bucket = self.table[index]
35        for pair in bucket:
36            if pair[0] == key:
37                pair[1] = value
38                return
39        bucket.append((key, value))
40
41    def _hash(self, key):
42        """
43        Hash a key to an index.
44        Returns:
45            int: Index of the bucket.
46        """
47        return hash(key) % self.size
48
49    def remove(self, key):
50        """
51        Remove a key-value pair from the hash table.
52        Parameters:
53            key: Key to remove.
54        Returns:
55            bool: True if removed successfully, False if key not found.
56        """
57        index = self._hash(key)
58        bucket = self.table[index]
59        for i, pair in enumerate(bucket):
60            if pair[0] == key:
61                del bucket[i]
62                return True
63        return False
64
65    def __str__(self):
66        return f"HashTable: {self.table}"
67
68    def __repr__(self):
69        return f"HashTable: {self.table}"
70
71    def __len__(self):
72        return len(self.table)
73
74    def __getitem__(self, key):
75        return self.table[key]
76
77    def __setitem__(self, key, value):
78        self.insert(key, value)
79
80    def __delitem__(self, key):
81        self.remove(key)
82
83    def __contains__(self, key):
84        return self.search(key)
85
86    def __iter__(self):
87        for bucket in self.table:
88            for pair in bucket:
89                yield pair
90
91    def __reversed__(self):
92        for bucket in reversed(self.table):
93            for pair in reversed(bucket):
94                yield pair
95
96    def __getitem__(self, key):
97        return self.table[key]
98
99    def __setitem__(self, key, value):
100        self.insert(key, value)
101
102    def __delitem__(self, key):
103        self.remove(key)
104
105    def __contains__(self, key):
106        return self.search(key)
107
108    def __iter__(self):
109        for bucket in self.table:
110            for pair in bucket:
111                yield pair
112
113    def __reversed__(self):
114        for bucket in reversed(self.table):
115            for pair in reversed(bucket):
116                yield pair
117
118    def __getitem__(self, key):
119        return self.table[key]
120
121    def __setitem__(self, key, value):
122        self.insert(key, value)
123
124    def __delitem__(self, key):
125        self.remove(key)
126
127    def __contains__(self, key):
128        return self.search(key)
129
130    def __iter__(self):
131        for bucket in self.table:
132            for pair in bucket:
133                yield pair
134
135    def __reversed__(self):
136        for bucket in reversed(self.table):
137            for pair in reversed(bucket):
138                yield pair
139
140    def __getitem__(self, key):
141        return self.table[key]
142
143    def __setitem__(self, key, value):
144        self.insert(key, value)
145
146    def __delitem__(self, key):
147        self.remove(key)
148
149    def __contains__(self, key):
150        return self.search(key)
151
152    def __iter__(self):
153        for bucket in self.table:
154            for pair in bucket:
155                yield pair
156
157    def __reversed__(self):
158        for bucket in reversed(self.table):
159            for pair in reversed(bucket):
160                yield pair
161
162    def __getitem__(self, key):
163        return self.table[key]
164
165    def __setitem__(self, key, value):
166        self.insert(key, value)
167
168    def __delitem__(self, key):
169        self.remove(key)
170
171    def __contains__(self, key):
172        return self.search(key)
173
174    def __iter__(self):
175        for bucket in self.table:
176            for pair in bucket:
177                yield pair
178
179    def __reversed__(self):
180        for bucket in reversed(self.table):
181            for pair in reversed(bucket):
182                yield pair
183
184    def __getitem__(self, key):
185        return self.table[key]
186
187    def __setitem__(self, key, value):
188        self.insert(key, value)
189
190    def __delitem__(self, key):
191        self.remove(key)
192
193    def __contains__(self, key):
194        return self.search(key)
195
196    def __iter__(self):
197        for bucket in self.table:
198            for pair in bucket:
199                yield pair
200
201    def __reversed__(self):
202        for bucket in reversed(self.table):
203            for pair in reversed(bucket):
204                yield pair
205
206    def __getitem__(self, key):
207        return self.table[key]
208
209    def __setitem__(self, key, value):
210        self.insert(key, value)
211
212    def __delitem__(self, key):
213        self.remove(key)
214
215    def __contains__(self, key):
216        return self.search(key)
217
218    def __iter__(self):
219        for bucket in self.table:
220            for pair in bucket:
221                yield pair
222
223    def __reversed__(self):
224        for bucket in reversed(self.table):
225            for pair in reversed(bucket):
226                yield pair
227
228    def __getitem__(self, key):
229        return self.table[key]
230
231    def __setitem__(self, key, value):
232        self.insert(key, value)
233
234    def __delitem__(self, key):
235        self.remove(key)
236
237    def __contains__(self, key):
238        return self.search(key)
239
240    def __iter__(self):
241        for bucket in self.table:
242            for pair in bucket:
243                yield pair
244
245    def __reversed__(self):
246        for bucket in reversed(self.table):
247            for pair in reversed(bucket):
248                yield pair
249
250    def __getitem__(self, key):
251        return self.table[key]
252
253    def __setitem__(self, key, value):
254        self.insert(key, value)
255
256    def __delitem__(self, key):
257        self.remove(key)
258
259    def __contains__(self, key):
260        return self.search(key)
261
262    def __iter__(self):
263        for bucket in self.table:
264            for pair in bucket:
265                yield pair
266
267    def __reversed__(self):
268        for bucket in reversed(self.table):
269            for pair in reversed(bucket):
270                yield pair
271
272    def __getitem__(self, key):
273        return self.table[key]
274
275    def __setitem__(self, key, value):
276        self.insert(key, value)
277
278    def __delitem__(self, key):
279        self.remove(key)
280
281    def __contains__(self, key):
282        return self.search(key)
283
284    def __iter__(self):
285        for bucket in self.table:
286            for pair in bucket:
287                yield pair
288
289    def __reversed__(self):
290        for bucket in reversed(self.table):
291            for pair in reversed(bucket):
292                yield pair
293
294    def __getitem__(self, key):
295        return self.table[key]
296
297    def __setitem__(self, key, value):
298        self.insert(key, value)
299
300    def __delitem__(self, key):
301        self.remove(key)
302
303    def __contains__(self, key):
304        return self.search(key)
305
306    def __iter__(self):
307        for bucket in self.table:
308            for pair in bucket:
309                yield pair
310
311    def __reversed__(self):
312        for bucket in reversed(self.table):
313            for pair in reversed(bucket):
314                yield pair
315
316    def __getitem__(self, key):
317        return self.table[key]
318
319    def __setitem__(self, key, value):
320        self.insert(key, value)
321
322    def __delitem__(self, key):
323        self.remove(key)
324
325    def __contains__(self, key):
326        return self.search(key)
327
328    def __iter__(self):
329        for bucket in self.table:
330            for pair in bucket:
331                yield pair
332
333    def __reversed__(self):
334        for bucket in reversed(self.table):
335            for pair in reversed(bucket):
336                yield pair
337
338    def __getitem__(self, key):
339        return self.table[key]
340
341    def __setitem__(self, key, value):
342        self.insert(key, value)
343
344    def __delitem__(self, key):
345        self.remove(key)
346
347    def __contains__(self, key):
348        return self.search(key)
349
350    def __iter__(self):
351        for bucket in self.table:
352            for pair in bucket:
353                yield pair
354
355    def __reversed__(self):
356        for bucket in reversed(self.table):
357            for pair in reversed(bucket):
358                yield pair
359
360    def __getitem__(self, key):
361        return self.table[key]
362
363    def __setitem__(self, key, value):
364        self.insert(key, value)
365
366    def __delitem__(self, key):
367        self.remove(key)
368
369    def __contains__(self, key):
370        return self.search(key)
371
372    def __iter__(self):
373        for bucket in self.table:
374            for pair in bucket:
375                yield pair
376
377    def __reversed__(self):
378        for bucket in reversed(self.table):
379            for pair in reversed(bucket):
380                yield pair
381
382    def __getitem__(self, key):
383        return self.table[key]
384
385    def __setitem__(self, key, value):
386        self.insert(key, value)
387
388    def __delitem__(self, key):
389        self.remove(key)
390
391    def __contains__(self, key):
392        return self.search(key)
393
394    def __iter__(self):
395        for bucket in self.table:
396            for pair in bucket:
397                yield pair
398
399    def __reversed__(self):
400        for bucket in reversed(self.table):
401            for pair in reversed(bucket):
402                yield pair
403
404    def __getitem__(self, key):
405        return self.table[key]
406
407    def __setitem__(self, key, value):
408        self.insert(key, value)
409
410    def __delitem__(self, key):
411        self.remove(key)
412
413    def __contains__(self, key):
414        return self.search(key)
415
416    def __iter__(self):
417        for bucket in self.table:
418            for pair in bucket:
419                yield pair
420
421    def __reversed__(self):
422        for bucket in reversed(self.table):
423            for pair in reversed(bucket):
424                yield pair
425
426    def __getitem__(self, key):
427        return self.table[key]
428
429    def __setitem__(self, key, value):
430        self.insert(key, value)
431
432    def __delitem__(self, key):
433        self.remove(key)
434
435    def __contains__(self, key):
436        return self.search(key)
437
438    def __iter__(self):
439        for bucket in self.table:
440            for pair in bucket:
441                yield pair
442
443    def __reversed__(self):
444        for bucket in reversed(self.table):
445            for pair in reversed(bucket):
446                yield pair
447
448    def __getitem__(self, key):
449        return self.table[key]
450
451    def __setitem__(self, key, value):
452        self.insert(key, value)
453
454    def __delitem__(self, key):
455        self.remove(key)
456
457    def __contains__(self, key):
458        return self.search(key)
459
460    def __iter__(self):
461        for bucket in self.table:
462            for pair in bucket:
463                yield pair
464
465    def __reversed__(self):
466        for bucket in reversed(self.table):
467            for pair in reversed(bucket):
468                yield pair
469
470    def __getitem__(self, key):
471        return self.table[key]
472
473    def __setitem__(self, key, value):
474        self.insert(key, value)
475
476    def __delitem__(self, key):
477        self.remove(key)
478
479    def __contains__(self, key):
480        return self.search(key)
481
482    def __iter__(self):
483        for bucket in self.table:
484            for pair in bucket:
485                yield pair
486
487    def __reversed__(self):
488        for bucket in reversed(self.table):
489            for pair in reversed(bucket):
490                yield pair
491
492    def __getitem__(self, key):
493        return self.table[key]
494
495    def __setitem__(self, key, value):
496        self.insert(key, value)
497
498    def __delitem__(self, key):
499        self.remove(key)
500
501    def __contains__(self, key):
502        return self.search(key)
503
504    def __iter__(self):
505        for bucket in self.table:
506            for pair in bucket:
507                yield pair
508
509    def __reversed__(self):
510        for bucket in reversed(self.table):
511            for pair in reversed(bucket):
512                yield pair
513
514    def __getitem__(self, key):
515        return self.table[key]
516
517    def __setitem__(self, key, value):
518        self.insert(key, value)
519
520    def __delitem__(self, key):
521        self.remove(key)
522
523    def __contains__(self, key):
524        return self.search(key)
525
526    def __iter__(self):
527        for bucket in self.table:
528            for pair in bucket:
529                yield pair
530
531    def __reversed__(self):
532        for bucket in reversed(self.table):
533            for pair in reversed(bucket):
534                yield pair
535
536    def __getitem__(self, key):
537        return self.table[key]
538
539    def __setitem__(self, key, value):
540        self.insert(key, value)
541
542    def __delitem__(self, key):
543        self.remove(key)
544
545    def __contains__(self, key):
546        return self.search(key)
547
548    def __iter__(self):
549        for bucket in self.table:
550            for pair in bucket:
551                yield pair
552
553    def __reversed__(self):
554        for bucket in reversed(self.table):
555            for pair in reversed(bucket):
556                yield pair
557
558    def __getitem__(self, key):
559        return self.table[key]
560
561    def __setitem__(self, key, value):
562        self.insert(key, value)
563
564    def __delitem__(self, key):
565        self.remove(key)
566
567    def __contains__(self, key):
568        return self.search(key)
569
570    def __iter__(self):
571        for bucket in self.table:
572            for pair in bucket:
573                yield pair
574
575    def __reversed__(self):
576        for bucket in reversed(self.table):
577            for pair in reversed(bucket):
578                yield pair
579
580    def __getitem__(self, key):
581        return self.table[key]
582
583    def __setitem__(self, key, value):
584        self.insert(key, value)
585
586    def __delitem__(self, key):
587        self.remove(key)
588
589    def __contains__(self, key):
590        return self.search(key)
591
592    def __iter__(self):
593        for bucket in self.table:
594            for pair in bucket:
595                yield pair
596
597    def __reversed__(self):
598        for bucket in reversed(self.table):
599            for pair in reversed(bucket):
600                yield pair
601
602    def __getitem__(self, key):
603        return self.table[key]
604
605    def __setitem__(self, key, value):
606        self.insert(key, value)
607
608    def __delitem__(self, key):
609        self.remove(key)
610
611    def __contains__(self, key):
612        return self.search(key)
613
614    def __iter__(self):
615        for bucket in self.table:
616            for pair in bucket:
617                yield pair
618
619    def __reversed__(self):
620        for bucket in reversed(self.table):
621            for pair in reversed(bucket):
622                yield pair
623
624    def __getitem__(self, key):
625        return self.table[key]
626
627    def __setitem__(self, key, value):
628        self.insert(key, value)
629
630    def __delitem__(self, key):
631        self.remove(key)
632
633    def __contains__(self, key):
634        return self.search(key)
635
636    def __iter__(self):
637        for bucket in self.table:
638            for pair in bucket:
639                yield pair
640
641    def __reversed__(self):
642        for bucket in reversed(self.table):
643            for pair in reversed(bucket):
644                yield pair
645
646    def __getitem__(self, key):
647        return self.table[key]
648
649    def __setitem__(self, key, value):
650        self.insert(key, value)
651
652    def __delitem__(self, key):
653        self.remove(key)
654
655    def __contains__(self, key):
656        return self.search(key)
657
658    def __iter__(self):
659        for bucket in self.table:
660            for pair in bucket:
661                yield pair
662
663    def __reversed__(self):
664        for bucket in reversed(self.table):
665            for pair in reversed(bucket):
666                yield pair
667
668    def __getitem__(self, key):
669        return self.table[key]
670
671    def __setitem__(self, key, value):
672        self.insert(key, value)
673
674    def __delitem__(self, key):
675        self.remove(key)
676
677    def __contains__(self, key):
678        return self.search(key)
679
680    def __iter__(self):
681        for bucket in self.table:
682            for pair in bucket:
683                yield pair
684
685    def __reversed__(self):
686        for bucket in reversed(self.table):
687            for pair in reversed(bucket):
688                yield pair
689
690    def __getitem__(self, key):
691        return self.table[key]
692
693    def __setitem__(self, key, value):
694        self.insert(key, value)
695
696    def __delitem__(self, key):
697        self.remove(key)
698
699    def __contains__(self, key):
700        return self.search(key)
701
702    def __iter__(self):
703        for bucket in self.table:
704            for pair in bucket:
705                yield pair
706
707    def __reversed__(self):
708        for bucket in reversed(self.table):
709            for pair in reversed(bucket):
710                yield pair
711
712    def __getitem__(self, key):
713        return self.table[key]
714
715    def __setitem__(self, key, value):
716        self.insert(key, value)
717
718    def __delitem__(self, key):
719        self.remove(key)
720
721    def __contains__(self, key):
722        return self.search(key)
723
724    def __iter__(self):
725        for bucket in self.table:
726            for pair in bucket:
727                yield pair
728
729    def __reversed__(self):
730        for bucket in reversed(self.table):
731            for pair in reversed(bucket):
732                yield pair
733
734    def __getitem__(self, key):
735        return self.table[key]
736
737    def __setitem__(self, key, value):
738        self.insert(key, value)
739
740    def __delitem__(self, key):
741        self.remove(key)
742
743    def __contains__(self, key):
744        return self.search(key)
745
746    def __iter__(self):
747        for bucket in self.table:
748            for pair in bucket:
749                yield pair
750
751    def __reversed__(self):
752        for bucket in reversed(self.table):
753            for pair in reversed(bucket):
754                yield pair
755
756    def __getitem__(self, key):
757        return self.table[key]
758
759    def __setitem__(self, key, value):
760        self.insert(key, value)
761
762    def __delitem__(self, key):
763        self.remove(key)
764
765    def __contains__(self, key):
766        return self.search(key)
767
768    def __iter__(self):
769        for bucket in self.table:
770            for pair in bucket:
771                yield pair
772
773    def __reversed__(self):
774        for bucket in reversed(self.table):
775            for pair in reversed(bucket):
776                yield pair
777
778    def __getitem__(self, key):
779        return self.table[key]
780
781    def __setitem__(self, key, value):
782        self.insert(key, value)
783
784    def __delitem__(self, key):
785        self.remove(key)
786
787    def __contains__(self, key):
788        return self.search(key)
789
790    def __iter__(self):
791        for bucket in self.table:
792            for pair in bucket:
793                yield pair
794
795    def __reversed__(self):
796        for bucket in reversed(self.table):
797            for pair in reversed(bucket):
798                yield pair
799
800    def __getitem__(self, key):
801        return self.table[key]
802
803    def __setitem__(self, key, value):
804        self.insert(key, value)
805
806    def __delitem__(self, key):
807        self.remove(key)
808
809    def __contains__(self, key):
810        return self.search(key)
811
812    def __iter__(self):
813        for bucket in self.table:
814            for pair in bucket:
815                yield pair
816
817    def __reversed__(self):
818        for bucket in reversed(self.table):
819            for pair in reversed(bucket):
820                yield pair
821
822    def __getitem__(self, key):
823        return self.table[key]
824
825    def __setitem__(self, key, value):
826        self.insert(key, value)
827
828    def __delitem__(self, key):
829        self.remove(key)
830
831    def __contains__(self, key):
832        return self.search(key)
833
834    def __iter__(self):
835        for bucket in self.table:
836            for pair in bucket:
837                yield pair
838
839    def __reversed__(self):
840        for bucket in reversed(self.table):
841            for pair in reversed(bucket):
842                yield pair
843
844    def __getitem__(self, key):
845        return self.table[key]
846
847    def __setitem__(self, key, value):
848        self.insert(key, value)
849
850    def __delitem__(self, key):
851        self.remove(key)
852
853    def __contains__(self, key):
854        return self.search(key)
855
856    def __iter__(self):
857        for bucket in self.table:
858            for pair in bucket:
859                yield pair
860
861    def __reversed__(self):
862        for bucket in reversed(self.table):
863            for pair in reversed(bucket):
864                yield pair
865
866    def __getitem__(self, key):
867        return self.table[key]
868
869    def __setitem__(self, key, value):
870        self.insert(key, value)
871
872    def __delitem__(self, key):
873        self.remove(key)
874
875    def __contains__(self, key):
876        return self.search(key)
877
878    def __iter__(self):
879        for bucket in self.table:
880            for pair in bucket:
881                yield pair
882
883    def __reversed__(self):
884        for bucket in reversed(self.table):
885            for pair in reversed(bucket):
886                yield pair
887
888    def __getitem__(self, key):
889        return self.table[key]
890
891    def __setitem__(self, key, value):
892        self.insert(key, value)
893
894    def __delitem__(self, key):
895        self.remove(key)
896
897    def __contains__(self, key):
898        return self.search(key)
899
900    def __iter__(self):
901        for bucket in self.table:
902            for pair in bucket:
903                yield pair
904
905    def __reversed__(self):
906        for bucket in reversed(self.table):
907            for pair in reversed(bucket):
908                yield pair
909
910    def __getitem__(self, key):
911        return self.table[key]
912
913    def __setitem__(self, key, value):
914        self.insert(key, value)
915
916    def __delitem__(self, key):
917        self.remove(key)
918
919    def __contains__(self, key):
920        return self.search(key)
921
922    def __iter__(self):
923        for bucket in self.table:
924            for pair in bucket:
925                yield pair
926
927    def __reversed__(self):
928        for bucket in reversed(self.table):
929            for pair in reversed(bucket):
930                yield pair
931
932    def __getitem__(self, key):
933        return self.table[key]
934
935    def __setitem__(self, key, value):
936        self.insert(key, value)
937
938    def __delitem__(self, key):
939        self.remove(key)
940
941    def __contains__(self, key):
942        return self.search(key)
943
944    def __iter__(self):
945        for bucket in self.table:
946            for pair in bucket:
947                yield pair
948
949    def __reversed__(self):
950        for bucket in reversed(self.table):
951            for pair in reversed(bucket):
952                yield pair
953
954    def __getitem__(self, key):
955        return self.table[key]
956
957    def __setitem__(self, key, value):
958        self.insert(key, value)
959
960    def __delitem__(self, key):
961        self.remove(key)
962
963    def __contains__(self, key):
964        return self.search(key)
965
966    def __iter__(self):
967        for bucket in self.table:
968            for pair in bucket:
969                yield pair
970
971    def __reversed__(self):
972        for bucket in reversed(self.table):
973            for pair in reversed(bucket):
974                yield pair
975
976    def __getitem__(self, key):
977        return self.table[key]
978
979    def __setitem__(self, key, value):
980        self.insert(key, value)
981
982    def __delitem__(self, key):
983        self.remove(key)
984
985    def __contains__(self, key):
986        return self.search(key)
987
988    def __iter__(self):
989        for bucket in self.table:
990            for pair in bucket:
991                yield pair
992
993    def __reversed__(self):
994        for bucket in reversed(self.table):
995            for pair in reversed(bucket):
996                yield pair
997
998    def __getitem__(self, key):
999        return self.table[key]
1000
1001    def __setitem__(self, key, value):
1002        self.insert(key, value)
1003
1004    def __delitem__(self, key):
1005        self.remove(key)
1006
1007    def __contains__(self, key):
1008        return self.search(key)
1009
1010    def __iter__(self):
1011        for bucket in self.table:
1012            for pair in bucket:
1013                yield pair
1014
1015    def __reversed__(self):
1016        for bucket in reversed(self.table):
1017            for pair in reversed(bucket):
1018                yield pair
1019
1020    def __getitem__(self, key):
1021        return self.table[key]
1022
1023    def __setitem__(self, key, value):
1024        self.insert(key, value)
1025
1026    def __delitem__(self, key):
1027        self.remove(key)
1028
1029    def __contains__(self, key):
1030        return self.search(key)
1031
1032    def __iter__(self):
1033        for bucket in self.table:
1034            for pair in bucket:
1035                yield pair
1036
1037    def __reversed__(self):
1038        for bucket in reversed(self.table):
1039            for pair in reversed(bucket):
1040                yield pair
1041
1042    def __getitem__(self, key):
1043        return self.table[key]
1044
1045    def __setitem__(self, key, value):
1046        self.insert(key, value)
1047
1048    def __delitem__(self, key):
1049        self.remove(key)
1050
1051    def __contains__(self, key):
1052        return self.search(key)
1053
1054    def __iter__(self):
1055        for bucket in self.table:
1056            for pair in bucket:
1057                yield pair
1058
1059    def __reversed__(self):
1060        for bucket in reversed(self.table):
1061            for pair in reversed(bucket):
1062                yield pair
1063
1064    def __getitem__(self, key):
1065        return self.table[key]
1066
1067    def __setitem__(self, key, value):
1068        self.insert(key, value)
1069
1070    def __delitem__(self, key):
1071        self.remove(key)
1072
1073    def __contains__(self, key):
1074        return self.search(key)
1075
1076    def __iter__(self):
1077        for bucket in self.table:
1078            for pair in bucket:
1079                yield pair
1080
1081    def __reversed__(self):
1082        for bucket in reversed(self.table):
1083            for pair in reversed(bucket):
1084                yield pair
1085
1086    def __getitem__(self, key):
1087        return self.table[key]
1088
1089    def __setitem__(self, key, value):
1090        self.insert(key, value)
1091
1092    def __delitem__(self, key):
1093        self.remove(key)
1094
1095    def __contains__(self, key):
1096        return self.search(key)
1097
1098    def __iter__(self):
1099        for bucket in self.table:
1100            for pair in bucket:
1101                yield pair
1102
1103    def __reversed__(self):
1104        for bucket in reversed(self.table):
1105            for pair in reversed(bucket):
1106                yield pair
1107
1108    def __getitem__(self, key):
1109        return self.table[key]
1110
1111    def __setitem__(self, key, value):
1112        self.insert(key, value)
1113
1114    def __delitem__(self, key):
1115        self.remove(key)
1116
1117    def __contains__(self, key):
1118        return self.search(key)
1119
1120    def __iter__(self):
1121        for bucket in self.table:
1122            for pair in bucket:
1123                yield pair
1124
1125    def __reversed__(self):
1126        for bucket in reversed(self.table):
1127            for pair in reversed(bucket):
1128                yield pair
1129
1130    def __getitem__(self, key):
1131        return self.table[key]
1132
1133    def __setitem__(self, key, value):
1134        self.insert(key, value)
1135
1136    def __delitem__(self, key):
1137        self.remove(key)
1138
1139    def __contains__(self, key):
1140        return self.search(key)
1141
1142    def __iter__(self):
1143        for bucket in self.table:
1144            for pair in bucket:
1145                yield pair
1146
1147    def __reversed__(self):
1148        for bucket in reversed(self.table):
1149            for pair in reversed(bucket):
1150                yield pair
1151
1152    def __getitem__(self, key):
1153        return self.table[key]
1154
1155    def __setitem__(self, key, value):
1156        self.insert(key, value)
1157
1158    def __delitem__(self, key):
1159        self.remove(key)
1160
1161    def __contains__(self, key):
1162        return self.search(key)
1163
1164    def __iter__(self):
1165        for bucket in self.table:
1166            for pair in bucket:
1167                yield pair
1168
1169    def __reversed__(self):
1170        for bucket in reversed(self.table):
1171            for pair in reversed(bucket):
1172                yield pair
1173
1174    def __getitem__(self, key):
1175        return self.table[key]
1176
1177    def __setitem__(self, key, value):
1178        self.insert(key, value)
1179
1180    def __delitem__(self, key):
1181        self.remove(key)
1182
1183    def __contains__(self, key):
1184        return self.search(key)
1185
1186    def __iter__(self):
1187        for bucket in self.table:
1188            for pair in bucket:
1189                yield pair
1190
1191    def __reversed__(self):
1192        for bucket in reversed(self.table):
1193            for pair in reversed(bucket):
1194                yield pair
1195
1196    def __getitem__(self, key):
1197        return self.table[key]
1198
1199    def __setitem__(self, key, value):
1200        self.insert(key, value)
1201
1202    def __delitem__(self, key):
1203        self.remove(key)
1204
1205    def __contains__(self, key):
1206        return self.search(key)
1207
1208    def __iter__(self):
1209        for bucket in self.table:
1210            for pair in bucket:
1211                yield pair
1212
1213    def __reversed__(self):
1214        for bucket in reversed(self.table):
1215            for pair in reversed(bucket):
1216                yield pair
1217
1218    def __getitem__(self, key):
1219        return self.table[key]
1220
1221    def __setitem__(self, key, value):
1222        self.insert(key, value)
1223
1224    def __delitem__(self, key):
1225        self.remove(key)
1226
1227    def __contains__(self, key):
1228        return self.search(key)
1229
1230    def __iter__(self):
1231        for bucket in self.table:
1232            for pair in bucket:
1233                yield pair
1234
1235    def __reversed__(self):
1236        for bucket in reversed(self.table):
1237            for pair in reversed(bucket):
1238                yield pair
1239
1240    def __getitem__(self, key):
1241        return self.table[key]
1242
1243    def __setitem__(self, key, value):
1244        self.insert(key, value)
1245
1246    def __delitem__(self, key):
1247        self.remove(key)
1248
1249    def __contains__(self, key):
1250        return self.search(key)
1251
1252    def __iter__(self):
1253        for bucket in self.table:
1254            for pair in bucket:
1255                yield pair
1256
1257    def __reversed__(self):
1258        for bucket in reversed(self.table):
1259            for pair in reversed(bucket):
1260                yield pair
1261
1262    def __getitem__(self, key):
1263        return self.table[key]
1264
1265    def __setitem__(self, key, value):
1266        self.insert(key, value)
1267
1268    def __delitem__(self, key):
1269        self.remove(key)
1270
1271    def __contains__(self, key):
1272        return self.search(key)
1273
1274    def __iter__(self):
1275        for bucket in self.table:
1276            for pair in bucket:
1277                yield pair
1278
1279    def __reversed__(self):
1280        for bucket in reversed(self.table):
1281            for pair in reversed(bucket):
1282                yield pair
1283
1284    def __getitem__(self, key):
1285        return self.table[key]
1286
1287    def __setitem__(self, key, value):
1288        self.insert(key, value)
1289
1290    def __delitem__(self, key):
1291        self.remove(key)
1292
1293    def __contains__(self, key):
1294        return self.search(key)
1295
1296    def __iter__(self):
1297        for bucket in self.table:
1298            for pair in bucket:
1299                yield pair
1300
1301    def __reversed__(self):
1302        for bucket in reversed(self.table):
1303            for pair in reversed(bucket):
1304                yield pair
1305
1306    def __getitem__(self, key):
1307        return self.table[key]
1308
1309    def __setitem__(self, key, value):
1310        self.insert(key, value)
1311
1312    def __delitem__(self, key):
1313        self.remove(key)
1314
1315    def __contains__(self, key):
1316        return self.search(key)
```

```
- """ Prompt:
2 Generate a Python class named Graph that represents a graph using an adjacency list.
3 - The class should include the following methods:
4 add_vertex(vertex) - to add a new vertex to the graph
5 add_edge(vertex1, vertex2) - to add an edge between two vertices
6 display() - to display all vertices and their connected neighbors. """

class Graph:
    """
    Graph implementation using an adjacency list.
    Attributes:
    - adj_list: Dictionary storing adjacent vertices
    - num_vertices: Number of vertices in the graph
    """
    def __init__(self):
        """Initialize an empty graph"""
        self.adj_list = {}
        self.num_vertices = 0

    def add_vertex(self, vertex):
        """Add a vertex to the graph"""
        if vertex not in self.adj_list:
            self.adj_list[vertex] = []
            self.num_vertices += 1

    def add_edge(self, vertex1, vertex2):
        """Add an edge between two vertices"""
        if vertex1 not in self.adj_list:
            self.add_vertex(vertex1)
        if vertex2 not in self.adj_list:
            self.add_vertex(vertex2)
        self.adj_list[vertex1].append(vertex2)
        self.adj_list[vertex2].append(vertex1)

    def display(self):
        """Display the adjacency list of the graph"""
        for vertex in self.adj_list:
            print(f"Vertex {vertex}: {self.adj_list[vertex]}")

graph = Graph()
graph.add_vertex(1)
graph.add_vertex(2)
graph.add_vertex(3)
graph.add_edge(1, 2)
graph.add_edge(2, 3)
graph.add_edge(3, 1)
graph.display()
```

Task Description #6: Smart Hospital Management System – Data Structure Selection

A hospital wants to develop a Smart Hospital Management System that handles:

1. Patient Check-In System – Patients are registered and treated in order of arrival.
2. Emergency Case Handling – Critical patients must be treated first.
3. Medical Records Storage – Fast retrieval of patient details using ID.
4. Doctor Appointment Scheduling – Appointments sorted by time.
5. Hospital Room Navigation – Represent connections between wards and rooms.

Student Task:

- For each feature, select the most appropriate data structure from the list below:

o Stack o Queue

o Priority

Queue o

Linked List

o Binary Search
Tree (BST)

o Graph o Hash

Table o Deque

- Justify your choice in 2–3 sentences per feature.
- Implement one selected feature as a working Python program with AI-assisted code generation.

Smart Hospital Management System – Data Structure Selection

1. Patient Check-In System Selected

Data Structure: Queue

Justification:

A Queue follows the First-In-First-Out (FIFO) principle. Patients are treated in the order they arrive, ensuring fairness. Therefore, a Queue is the most suitable structure for managing regular patient check-ins.

2. Emergency Case Handling

Selected Data Structure: Priority Queue

Justification:

Emergency patients must be treated based on severity rather than arrival time. A Priority Queue allows higher-priority (critical) patients to be treated first. This ensures immediate attention to life-threatening cases.

3. Medical Records Storage Selected Data Structure: Hash

Table

Justification:

Patient records need fast retrieval using a unique patient ID. A Hash Table provides average $O(1)$ time complexity for search, insert, and delete operations. This makes record management efficient and scalable.

Appointments must be maintained in sorted order based on time. A Binary Search Tree automatically keeps elements sorted. This makes scheduling and time-based retrieval efficient.

Hospital rooms and wards can be represented as nodes, and pathways as edges. A Graph effectively models connections between locations. It also supports pathfinding for navigation within the hospital.

[illegible]

1. Traffic Signal Queue – Vehicles waiting at signals.
2. Emergency Vehicle Priority Handling – Ambulances and fire trucks prioritized.
3. Vehicle Registration Lookup – Instant access to vehicle details.
4. Road Network Mapping – Roads and intersections connected logically.
5. Parking Slot Availability – Track available and occupied slots.

Student Task

- For each feature, select the most appropriate data structure from the list below:

o Stack o Queue

o Priority Queue o

Linked List

o Binary Search

Tree (BST) o Graph o

Hash Table o Deque

- Justify your choice in 2–3 sentences per feature.
- Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

- A table mapping feature → chosen data structure → justification.
- A functional Python program implementing the chosen feature with comments and docstrings.

Smart City Traffic Control System

1. Traffic Signal Queue Selected Data

Structure: Queue

Justification:

Vehicles waiting at a traffic signal follow the First-In-First-Out (FIFO) principle. The vehicle that arrives first should move first when the signal turns green. Therefore, a Queue is the most appropriate structure for managing traffic signals fairly and efficiently.

2. Emergency Vehicle Priority Handling

Selected Data Structure: Priority Queue

Justification:

Emergency vehicles such as ambulances and fire trucks must be given immediate access regardless of arrival time. A Priority Queue allows higher-priority vehicles to be

processed before regular vehicles. This ensures faster emergency response and improved public safety.

3. Vehicle Registration Lookup

Selected Data Structure: Hash Table

Justification:

Vehicle details must be accessed instantly using a registration number. A Hash Table provides average $O(1)$ time complexity for search operations. This makes it ideal for fast and efficient vehicle information retrieval.

4. Road Network Mapping Selected

Data Structure: Graph

Justification:

Roads and intersections can be represented as nodes and edges. A Graph efficiently models connections between locations and supports route-finding algorithms. This makes it suitable for navigation and traffic management systems.

5. Parking Slot Availability

Selected Data Structure: Binary Search Tree (BST)

Justification:

Parking slots can be stored in sorted order based on slot numbers. A Binary Search Tree maintains elements in sorted order and allows efficient insertion, deletion, and searching. This makes it suitable for tracking available and occupied parking slots.

```
main.py
1 class Stack:
2     """
3     Stack implementation using Python list.
4     follows LIFO (Last-In-First-Out) principle. """
5     def __init__(self):
6         """
7         Initialize an empty stack.
8         """
9         self.items = []
10    def push(self, item):
11        """
12        Add an element to the top of the stack.
13        Parameters:
14            item: Element to be added
15        """
16        self.items.append(item)
17    def pop(self):
18        """
19        Remove and return the top element of the stack.
20        Returns:
21            The top element of the stack.
22        Raises:
23            IndexError: If stack is empty. """
24        if self.is_empty():
25            raise IndexError("Pop from empty stack")
26        return self.items.pop()
27    def peek(self):
28        """
29        Return the top element without removing it.
30        Returns:
31            The top element of the stack.
32        Raises:
33            IndexError: If stack is empty. """
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
101
102
103
104
105
106
107
108
109
110
111
112
113
114
115
116
117
118
119
120
121
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
1000
1001
1002
1003
1004
1005
1006
1007
1008
1009
1010
1011
1012
1013
1014
1015
1016
1017
1018
1019
1020
1021
1022
1023
1024
1025
1026
1027
1028
1029
1030
1031
1032
1033
1034
1035
1036
1037
1038
1039
1040
1041
1042
1043
1044
1045
1046
1047
1048
1049
1050
1051
1052
1053
1054
1055
1056
1057
1058
1059
1060
1061
1062
1063
1064
1065
1066
1067
1068
1069
1070
1071
1072
1073
1074
1075
1076
1077
1078
1079
1080
1081
1082
1083
1084
1085
1086
1087
1088
1089
1090
1091
1092
1093
1094
1095
1096
1097
1098
1099
1100
1101
1102
1103
1104
1105
1106
1107
1108
1109
1110
1111
1112
1113
1114
1115
1116
1117
1118
1119
1120
1121
1122
1123
1124
1125
1126
1127
1128
1129
1130
1131
1132
1133
1134
1135
1136
1137
1138
1139
1140
1141
1142
1143
1144
1145
1146
1147
1148
1149
1150
1151
1152
1153
1154
1155
1156
1157
1158
1159
1160
1161
1162
1163
1164
1165
1166
1167
1168
1169
1170
1171
1172
1173
1174
1175
1176
1177
1178
1179
1180
1181
1182
1183
1184
1185
1186
1187
1188
1189
1190
1191
1192
1193
1194
1195
1196
1197
1198
1199
1200
1201
1202
1203
1204
1205
1206
1207
1208
1209
1210
1211
1212
1213
1214
1215
1216
1217
1218
1219
1220
1221
1222
1223
1224
1225
1226
1227
1228
1229
1230
1231
1232
1233
1234
1235
1236
1237
1238
1239
1240
1241
1242
1243
1244
1245
1246
1247
1248
1249
1250
1251
1252
1253
1254
1255
1256
1257
1258
1259
1260
1261
1262
1263
1264
1265
1266
1267
1268
1269
1270
1271
1272
1273
1274
1275
1276
1277
1278
1279
1280
1281
1282
1283
1284
1285
1286
1287
1288
1289
1290
1291
1292
1293
1294
1295
1296
1297
1298
1299
1300
1301
1302
1303
1304
1305
1306
1307
1308
1309
1310
1311
1312
1313
1314
1315
1316
1317
1318
1319
1320
1321
1322
1323
1324
1325
1326
1327
1328
1329
1330
1331
1332
1333
1334
1335
1336
1337
1338
1339
1340
1341
1342
1343
1344
1345
1346
1347
1348
1349
1350
1351
1352
1353
1354
1355
1356
1357
1358
1359
1360
1361
1362
1363
1364
1365
1366
1367
1368
1369
1370
1371
1372
1373
1374
1375
1376
1377
1378
1379
1380
1381
1382
1383
1384
1385
1386
1387
1388
1389
1390
1391
1392
1393
1394
1395
1396
1397
1398
1399
1400
1401
1402
1403
1404
1405
1406
1407
1408
1409
1410
1411
1412
1413
1414
1415
1416
1417
1418
1419
1420
1421
1422
1423
1424
1425
1426
1427
1428
1429
1430
1431
1432
1433
1434
1435
1436
1437
1438
1439
1440
1441
1442
1443
1444
1445
1446
1447
1448
1449
1450
1451
1452
1453
1454
1455
1456
1457
1458
1459
1460
1461
1462
1463
1464
1465
1466
1467
1468
1469
1470
1471
1472
1473
1474
1475
1476
1477
1478
1479
1480
1481
1482
1483
1484
1485
1486
1487
1488
1489
1490
1491
1492
1493
1494
1495
1496
1497
1498
1499
1500
1501
1502
1503
1504
1505
1506
1507
1508
1509
1510
1511
1512
1513
1514
1515
1516
1517
1518
1519
1520
1521
1522
1523
1524
1525
1526
1527
1528
1529
1530
1531
1532
1533
1534
1535
1536
1537
1538
1539
1540
1541
1542
1543
1544
1545
1546
1547
1548
1549
1550
1551
1552
1553
1554
1555
1556
1557
1558
1559
1560
1561
1562
1563
1564
1565
1566
1567
1568
1569
1570
1571
1572
1573
1574
1575
1576
1577
1578
1579
1580
1581
1582
1583
1584
1585
1586
1587
1588
1589
1590
1591
1592
1593
1594
1595
1596
1597
1598
1599
1600
1601
1602
1603
1604
1605
1606
1607
1608
1609
1610
1611
1612
1613
1614
1615
1616
1617
1618
1619
1620
1621
1622
1623
1624
1625
1626
1627
1628
1629
1630
1631
1632
1633
1634
1635
1636
1637
1638
1639
1640
1641
1642
1643
1644
1645
1646
1647
1648
1649
1650
1651
1652
1653
1654
1655
1656
1657
1658
1659
1660
1661
1662
1663
1664
1665
1666
1667
1668
1669
1670
1671
1672
1673
1674
1675
1676
1677
1678
1679
1680
1681
1682
1683
1684
1685
1686
1687
1688
1689
1690
1691
1692
1693
1694
1695
1696
1697
1698
1699
1700
1701
1702
1703
1704
1705
1706
1707
1708
1709
1710
1711
1712
1713
1714
1715
1716
1717
1718
1719
1720
1721
1722
1723
1724
1725
1726
1727
1728
1729
1730
1731
1732
1733
1734
1735
1736
1737
1738
1739
1740
1741
1742
1743
1744
1745
1746
1747
1748
1749
1750
1751
1752
1753
1754
1755
1756
1757
1758
1759
1760
1761
1762
1763
1764
1765
1766
1767
1768
1769
1770
1771
1772
1773
1774
1775
1776
1777
1778
1779
1780
1781
1782
1783
1784
1785
1786
1787
1788
1789
1790
1791
1792
1793
1794
1795
1796
1797
1798
1799
1800
1801
1802
1803
1804
1805
1806
1807
1808
1809
1810
1811
1812
1813
1814
1815
1816
1817
1818
1819
1820
1821
1822
1823
1824
1825
1826
1827
1828
1829
1830
1831
1832
1833
1834
1835
1836
1837
1838
1839
1840
1841
1842
1843
1844
1845
1846
1847
1848
1849
1850
1851
1852
1853
1854
1855
1856
1857
1858
1859
1860
1861
1862
1863
1864
1865
1866
1867
1868
1869
1870
1871
1872
1873
1874
1875
1876
1877
1878
1879
1880
1881
1882
1883
1884
1885
1886
1887
1888
1889
1890
1891
1892
1893
1894
1895
1896
1897
1898
1899
1900
1901
1902
1903
1904
1905
1906
1907
1908
1909
1910
1911
1912
1913
1914
1915
1916
1917
1918
1919
1920
1921
1922
1923
1924
1925
1926
1927
1928
1929
1930
1931
1932
1933
1934
1935
1936
1937
1938
1939
1940
1941
1942
1943
1944
1945
1946
1947
1948
1949
1950
1951
1952
1953
1954
1955
1956
1957
1958
1959
1960
1961
1962
1963
1964
1965
1966
1967
1968
1969
1970
1971
1972
1973
1974
1975
1976
1977
1978
1979
1980
1981
1982
1983
1984
1985
1986
1987
1988
1989
1990
1991
1992
1993
1994
1995
1996
1997
1998
1999
2000
2001
2002
2003
2004
2005
2006
2007
2008
2009
2010
2011
2012
2013
2014
2015
2016
2017
2018
2019
2020
2021
2022
2023
2024
2025
2026
2027
2028
2029
2030
2031
2032
2033
2034
2035
2036
2037
2038
2039
2040
2041
2042
2043
2044
2045
2046
2047
2048
2049
2050
2051
2052
2053
2054
2055
2056
2057
2058
2059
2060
2061
2062
2063
2064
2065
2066
2067
2068
2069
2070
2071
2072
2073
2074
2075
2076
2077
2078
2079
2080
2081
2082
2083
2084
2085
2086
2087
2088
2089
2090
2091
2092
2093
2094
2095
2096
2097
2098
2099
2100
2101
2102
2103
2104
2105
2106
2107
2108
2109
2110
2111
2112
2113
2114
2115
2116
2117
2118
2119
2120
2121
2122
2123
2124
2125
2126
2127
2128
2129
2130
2131
2132
2133
2134
2135
2136
2137
2138
2139
2140
2141
2142
2143
2144
2145
2146
2147
2148
2149
2150
2151
2152
2153
2154
2155
2156
2157
2158
2159
2160
2161
2162
2163
2164
2165
2166
2167
2168
2169
2170
2171
2172
2173
2174
2175
2176
2177
2178
2179
2180
2181
2182
2183
2184
2185
2186
2187
2188
2189
2190
2191
2192
2193
2194
2195
2196
2197
2198
2199
2200
2201
2202
2203
2204
2205
2206
2207
2208
2209
2210
2211
2212
2213
2214
2215
2216
2217
2218
2219
2220
2221
2222
2223
2224
2225
2226
2227
2228
2229
2230
2231
2232
2233
2234
2235
2236
2237
2238
2239
2240
2241
2242
2243
2244
2245
2246
2247
2248
2249
2250
2251
2252
2253
2254
2255
2256
2257
2258
2259
2260
2261
2262
2263
2264
2265
2266
2267
2268
2269
2270
2271
2272
2273
2274
2275
2276
2277
2278
2279
2280
2281
2282
2283
2284
2285
2286
2287
2288
2289
2290
2291
2292
2293
2294
2295
2296
2297
2298
2299
2300
2301
2302
2303
2304
2305
2306
2307
2308
2309
2310
2311
2312
2313
2314
2315
2316
2317
2318
2319
2320
2321
2322
2323
2324
2325
2326
2327
2328
2329
2330
2331
2332
2333
2334
2335
2336
2337
2338
2339
2340
2341
2342
2343
2344
2345
2346
2347
2348
2349
2350
2351
2352
2353
2354
2355
2356
2357
2358
2359
2360
2361
2362
2363
2364
2365
2366
2367
2368
2369
2370
2371
2372
2373
2374
2375
2376
2377
2378
2379
2380
2381
2382
2383
2384
2385
2386
2387
2388
2389
2390
2391
2392
2393
2394
2395
2396
2397
2398
2399
2400
2401
2402
2403
2404
2405
2406
2407
2408
2409
2410
2411
2412
2413
2414
2415
2416
2417
2418
2419
2420
2421
2422
2423
2424
2425
2426
2427
2428
2429
2430
2431
2432
2433
2434
2435
2436
2437
2438
2439
2440
2441
2442
2443
2444
2445
2446
2447
2448
2449
2450
2451
2452
2453
2454
2455
2456
2457
2458
2459
2460
2461
2462
2463
2464
2465
2466
2467
2468
2469
2470
2471
2472
2473
2474
2475
2476
2477
2478
2479
2480
2481
2482
2483
2484
2485
2486
2487
2488
2489
2490
2491
2492
2493
2494
2495
2496
2497
2498
2499
2500
2501
2502
2503
2504
2505
2506
2507
2508
2509
2510
2511
2512
2513
2514
2515
2516
2517
2518
2519
2520
2521
2522
2523
2524
2525
2526
2527
2528
2529
2530
2531
2532
2533
2534
2535
2536
2537
2538
2539
2540
2541
2542
2543
2544
2545
2546
2547
2548
2549
2550
2551
2552
2553
2554
2555
2556
2557
2558
2559
2560
2561
2562
2563
2564
2565
2566
2567
2568
2569
2570
2571
2572
2573
2574
2575
2576
2577
2578
2579
2580
2581
2582
2583
2584
2585
2586
2587
2588
2589
2590
2591
2592
2593
2594
2595
2596
2597
2598
2599
2600
2601
2602
2603
2604
2605
2606
2607
2608
2609
2610
2611
2612
2613
2614
2615
2616
2617
2618
2619
2620
2621
2622
2623
2624
2625
2626
2627
2628
2629
2630
2631
2632
2633
2634
2635
2636
2637
2638
2639
2640
2641
2642
2643
2644
2645
2646
2647
2648
2649
2650
2651
2652
2653
2654
26
```

Student Task

- For each feature, select the most appropriate data structure from the list below:

o Stack o Queue

o Priority Queue o

Linked List

o Binary Search

Tree (BST) o Graph o

Hash Table o Deque

- Justify your choice in 2–3 sentences per feature.
- Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

- A table mapping feature → chosen data structure → justification.
- A functional Python program implementing the chosen feature with comments and docstrings.

Smart E-Commerce Platform – Data Structure Selection

1. Shopping Cart Management Selected

Data Structure: Linked List

Justification:

Products in a shopping cart are added and removed dynamically. A Linked List allows efficient insertion and deletion without shifting elements. This makes it suitable for managing frequently changing cart items.

2. Order Processing System Selected

Data Structure: Queue

Justification:

Orders must be processed in the order they are placed. A Queue follows the First-InFirst-Out (FIFO) principle, ensuring fairness. Therefore, it is ideal for handling customer orders.

3. Top-Selling Products Tracker

Selected Data Structure: Priority

Queue

Justification:

Products need to be ranked based on sales count. A Priority Queue allows items with higher sales to be accessed first. This ensures efficient tracking of top-selling products.

4. Product Search Engine

Selected Data Structure: Hash Table

Justification:

Products are searched using unique product IDs. A Hash Table provides average $O(1)$ time complexity for search operations. This makes product lookup fast and efficient.

5. Delivery Route Planning Selected

Data Structure: Graph

Justification:

Warehouses and delivery locations can be represented as nodes, and roads as edges. A Graph efficiently models connections and supports shortest path algorithms. This makes it suitable for route optimization and logistics planning.

```
main.py                                     Run Output
1 # Import the random module
2 # Import the random module again, using from syntax
3 # Create a list to store product details
4 products = []
5 # Function to add a product
6 def add_product(product_id, name, price):
7     product = {'id': product_id,
8               'name': name,
9               'price': price}
10    products.append(product)
11    print(f"Product added successfully.")
12 # Function to search for a product
13 def search_products(product_id):
14     if product_id in products:
15         product = products[product_id]
16         print(f"Product ID: {product_id}")
17         print(f"Name: {product['name']}")
18         print(f"Price: {product['price']}")
19     else:
20         print(f"Product not found.")
21 # Function to display all products
22 def display_products():
23     if len(products) == 0:
24         print("No products available.")
25     else:
26         print("Available Products:")
27         for product in products:
28             print(f"ID: {product['id']}, Name: {product['name']}, Price: {product['price']}")
29 # Main function
30 def main():
31     add_product(001, 'Laptop', 12000)
32     add_product(002, 'Mobile', 2000)
33     add_product(003, 'Headphone', 3000)
34     products = display_products()
35     print("\nSearching for Product 001:")
36     search_products(001)
37     print("\nSearching for Product 002:")
38     search_products(002)
```